

Instituto Tecnológico de Costa Rica Área Académica de Ingeniería Mecatrónica Programa de Licenciatura en Ingeniería Mecatrónica Curso: MT-7003 Microprocesadores y microcontroladores Profesor: Rodolfo Piedra Camacho Semestre: I, 2026	Examen Ensamblador Fecha: 10 de abril del 2026 Tiempo: 3:00 horas Valor: 150 puntos
--	--

Instrucciones:

Las instrucciones generales del examen ensamblador son las estipuladas en el programa del curso. A esta se les suman las siguientes instrucciones específicas:

- Usted posee 3 horas para completar el examen
- El examen se realiza en trios.
- Adjunto a este enunciado puede encontrar archivos con el “header del ejercicio”, el código ya escrito no puede ser modificado y plantea las condiciones básicas del ejercicio a realizar. Modificar este código inicial invalida el ejercicio con una nota de 0.
- Usted debe de realizar un diagrama de flujo que explique su algoritmo de solución. Este puede ser hecho a mano siempre y cuando sea legible.
- Las soluciones deben de ser ejecutables en el simulador VisUAL2.
- Deben subir al TEC digital en la sección de evaluaciones un archivo comprimido que contenga **una carpeta con solo 6 archivos**. De entregar más de 6 archivos se descuentan 10 puntos de la nota final del examen:
 - El archivo con la solución de ensamblador. Uno por ejercicio.
 - El diagrama de flujo. Uno por ejercicio.
 - La carpeta debe de tener los apellidos de los estudiantes.
- Para el diagrama de flujo si necesita escanear múltiples imágenes puede hacerlo siempre y cuando todas estén en un único documento.
- La hora de entrega final del TEC Digital contempla dificultades de comunicación por el medio virtual, el examen original está pensado para 2 horas y media. Si el TEC digital no se encuentra disponible los estudiantes pueden enviar el archivo comprimido al correo del profesor. Entregas tardías implican una nota de 0.
- El uso de herramientas de inteligencia artificial (locales, en línea o de cualquier tipo) está prohibido durante la duración del examen o durante la defensa. En caso de usarlas se califica el examen con una nota de 0.
- La defensa del examen se agenda en el Excel del siguiente link: <https://docs.google.com/spreadsheets/d/1XfIF0BgZbzUbdQzApxXR0CdA8Fho3ABJ5R9N8DAy28/edit#gid=419884248>
 - Las citas poseen una duración de hasta 60 minutos máximo.
 - Las citas se realizan de forma presencial en el aula F304, o bien en el aula que el profesor avise.
 - La solicitud de cita se acepta únicamente por inscripción al Excel.
 - En caso de no agendar defensa para las 11:45pm del 10 de abril se procederá con una calificación binaria de la solución entregada y la pérdida de los puntos de defensa.

- El proceso de defensa se basa en:
 - Explicar los diagramas de flujo.
 - Ejecutar el código con la solución o bien las secciones de código funcionales.
 - Cada estudiante solo puede defender un ejercicio. Esto no aplica en caso de que existan parejas.
- La distribución de puntos por cada ejercicio respeta esta proporción (Ejemplo para ejercicio de 50 puntos):
 - Diagrama de Flujo: 10
 - Comentarios en código: 5
 - Funcionalidad: 15
 - Capacidad de Explicar el Código: 20
 - NOTA: Sacar 0/20 en la defensa provoca que el ejercicio entero obtenga un 0/50

I Ejercicio: Encriptación con Permutación. (50 puntos)

El cifrado Feistel consiste en los siguientes pasos:

1. Separar la palabra a trabajar en parte alta (L_0) y parte baja (R_0). Dividiendo la palabra original a la mitad.
2. En cada ronda ($n = 1$ hasta $n = X$) se ejecuta lo siguiente:
 - a. $L_n = R_{n-1}$
 - b. $R_n = L_{n-1} \text{ XOR } F(R_{n-1}, \text{Tabla})$
3. La palabra cifrada se obtiene concatenando L_X y R_X (Parte alta y baja respectivamente)

La función F es una permutación que reordena los bits del primer parámetro según una tabla de 16 bytes que vive en memoria. El byte en la posición Z (0-15) de la tabla indica de qué posición de bit del valor original se toma el bit Z del resultado.

Ejemplo: si la tabla contiene [5, 0, 15, 12, ...], entonces:

- Bit $Z=0$ del resultado es el Bit 5 del valor original
- Bit $Z=1$ del resultado es el Bit 0 del valor original
- Bit $Z=2$ del resultado es el Bit 15 del valor original
- Bit $Z=3$ del resultado es el Bit 12 del valor original
- ... y así para las 16 posiciones.

Usted debe de programar en ensamblador un Cifrado tipo Feistel de $X=4$ rondas.

Contemple las siguientes limitaciones.

- En $R2$ se encuentra la palabra a cifrar
- La tabla de permutación se encuentra a partir de la dirección $0x00FF0000$
- La tabla de permutaciones esta comprimida de tal forma que solo consume 4 palabras en memoria.
- El LSB del rango de memoria de la tabla de permutación es $Z = 0$, el MSB es $Z = 15$. Lógica Little endian.

El archivo header de este ejercicio posee solo una variante del ejercicio, durante la revisión el profesor modificara este header para asegurar todas las variantes.

II Ejercicio: Detector de Errores de Codificación. (50 puntos)

Considere un espacio de memoria definido. En este espacio de memoria usted encuentra instrucciones de ensamblador para una arquitectura ARM 32, igual a la vista en clase, usted debe de crear un código capaz de detectar si las instrucciones en memoria son válidas o no.

Por válida se entiende que respetan el ISA de ARM32 para instrucciones de memoria, aritmeticológicas o de control de flujo; dicho de otra forma, que el binario se puede traducir a una única instrucción ensamblador sin ambigüedad. Usted debe de contar cuantas palabras del espacio de memoria estan correctas y cuantas estan mal.

Para esto contemple las siguientes limitaciones:

- El registro R0 contiene la dirección inicial del espacio de memoria
- El registro R1 contiene la dirección final del espacio de memoria (inclusive)
- El registro R2 debe de almacenar la cantidad de instrucciones correctas
- El registro R3 debe de almacenar la cantidad de instrucciones incorrectas

El archivo header de este ejercicio posee solo una variante del ejercicio, durante la revisión el profesor modificara este header para asegurar todas las variantes

III Ejercicio: Desfragmentación de Memoria (50 puntos)

Considere un espacio de memoria que representa un “heap” simplificado. El bloque contiene N palabras donde cada palabra puede ser un **dato válido** o un **espacio vacío**. Las palabras vacías se identifican porque contienen el valor 0xDEADDEAD, las palabras válidas contienen cualquier otro valor.

Los datos válidos y los espacios vacíos están intercalados en un orden arbitrario. Usted debe **desfragmentar** este bloque: mover todos los datos válidos al inicio del bloque (conservando su orden relativo original) y todos los espacios vacíos al final.

Para esto contemple las siguientes limitaciones:

- R0: dirección inicial del bloque de memoria.
- R1: dirección final del bloque de memoria (inclusive).
- No se dispone de un bloque auxiliar en memoria. La reorganización debe ser in-place.
- No puede asumir ningún patrón en la fragmentación

El archivo header de este ejercicio posee solo una variante del ejercicio, durante la revisión el profesor modificará este header para asegurar todas las variantes.